



EVALUACIÓN PARCIAL N° 1

Automatización Análisis de Calidad y Seguridad

AUY1105 — Infraestructura como Código II

Campo	Detalle
Asignatura	Infraestructura como Código II
Sigla	AUY1105
Grupo	AM-GL
Integrante 1	Alonso Mieres (alonsoM10)
Integrante 2	Giancarlos Loaiza
Repositorio	https://github.com/alonsoM10/AUY1105-grupo-AM-GL
Fecha	28 de Abril de 2026
Profesor	Andres Galvez

1. Introducción.....	3
2. Requerimiento 1 — Repositorio de Código.....	5
3. Requerimiento 2 — Código de Infraestructura Terraform.....	8
3.3 main.tf — Recursos de Infraestructura.....	10
4. Requerimiento 3 — Automatización con GitHub Actions.....	13
5. Requerimiento 4 — Definición de Políticas OPA.....	16
6. Requerimiento 5 — Buenas Prácticas de Revisión de Código.....	18
6.3 Descripción del Flujo de Trabajo.....	19
Evidencia de AWS en ejecución.....	19

1. Introducción

1.1 Descripción General

La presente documentación corresponde al desarrollo de la Evaluación Parcial N°1 de la asignatura Infraestructura como Código II (AUY1105) del Instituto Profesional Duoc UC. El objetivo principal de esta evaluación fue implementar un pipeline automatizado mediante GitHub Actions, integrando herramientas modernas de análisis de calidad, seguridad y prácticas de infraestructura como código (IaC).

Todo el trabajo fue desarrollado en pareja por Alonso Mieres y Giancarlos Loaiza, utilizando el repositorio GitHub AUY1105-grupo-AM-GL como plataforma de versionamiento y colaboración, siguiendo las mejores prácticas de desarrollo con Pull Requests documentados.

1.2 Herramientas y Tecnologías Utilizadas

Terraform

Terraform es una herramienta de Infraestructura como Código (IaC) desarrollada por HashiCorp que permite definir, provisionar y gestionar infraestructura en múltiples proveedores de nube mediante archivos de configuración declarativos en lenguaje HCL (HashiCorp Configuration Language). En este proyecto se utilizó Terraform v1.14.9 para definir los recursos de infraestructura en AWS, incluyendo VPC, Subredes, Security Groups e instancias EC2.

GitHub Actions

GitHub Actions es una plataforma de integración y entrega continua (CI/CD) integrada directamente en GitHub. Permite automatizar flujos de trabajo mediante archivos YAML que definen eventos disparadores y jobs a ejecutar. En este proyecto se implementó un workflow que se activa automáticamente en cada Pull Request hacia la rama main, ejecutando análisis de calidad y seguridad del código Terraform.

TFLint

TFLint es una herramienta de análisis estático (linting) específica para código Terraform. Identifica errores de configuración, desviaciones de las mejores prácticas y posibles problemas de seguridad antes de que el código sea aplicado. Actúa como primera línea de defensa en el pipeline de CI/CD, garantizando la calidad del código de infraestructura.

Checkov

Checkov es una herramienta de análisis de seguridad estática para código de infraestructura como código. Escanea el código Terraform en busca de configuraciones incorrectas de seguridad, vulnerabilidades y desviaciones de las mejores prácticas de la industria, como el CIS Benchmark y otras normas de cumplimiento. Se ejecuta como segunda etapa del pipeline, asegurando que la infraestructura cumpla con los estándares de seguridad.

Open Policy Agent (OPA)

Open Policy Agent es un motor de políticas de propósito general de código abierto que permite definir y aplicar políticas de seguridad como código. En este proyecto se utilizó OPA con el lenguaje Rego para implementar dos políticas de seguridad: una que prohíbe el acceso SSH público y otra que restringe el tipo de instancia EC2 a t2.micro, garantizando el cumplimiento de las políticas de seguridad definidas.

AWS (Amazon Web Services)

Amazon Web Services es la plataforma de nube pública utilizada para el despliegue de la infraestructura. Se utilizó a través de AWS Academy Learner Lab, que proporciona credenciales temporales para acceder a los servicios de AWS. Los recursos desplegados incluyen VPC, Subredes, Security Groups e instancias EC2 en la región us-east-1.

2. Requerimiento 1 — Repositorio de Código

Se configuró el repositorio GitHub AUY1105-grupo-AM-GL con todos los archivos base requeridos por la evaluación. El repositorio almacena tanto el código Terraform de la infraestructura como el workflow de automatización GitHub Actions y las políticas de seguridad OPA.

2.1 Estructura del Repositorio

Archivo / Carpeta	Descripción
README.md	Documentación principal: objetivos, integrantes, estructura e instrucciones de uso
.gitignore	Excluye archivos Terraform (.terraform/, *.tfstate, *.pem, secrets/)
CHANGELOG.md	Registro histórico de todos los cambios del proyecto
terraform/	Código IaC: main.tf, providers.tf, variables.tf, outputs.tf
.github/workflows/	Workflow de automatización: terraform-ci.yml
policias/opa/	Políticas de seguridad OPA en lenguaje Rego

2.2 Evidencia — Repositorio GitHub

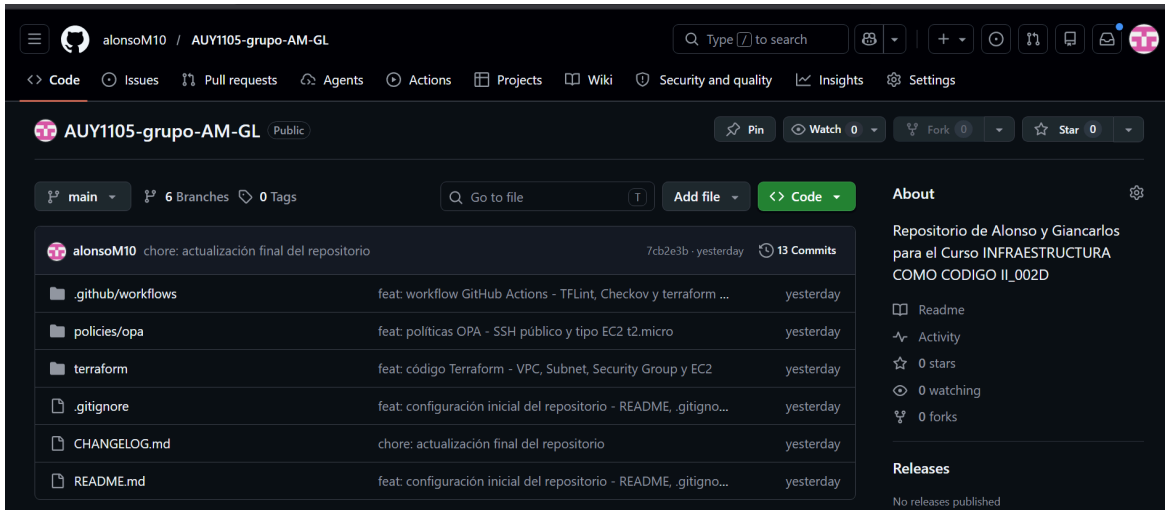


Figura 1: Repositorio AUY1105-grupo-AM-GL en GitHub con estructura completa

2.3 Evidencia — README.md

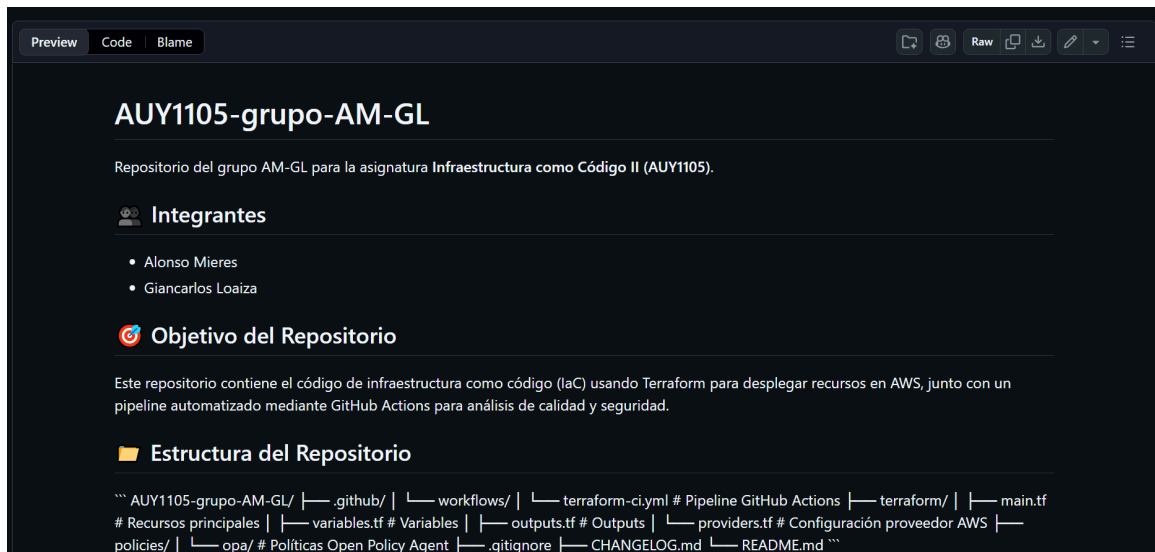


Figura 2: README.md con descripción del proyecto, integrantes y estructura del repositorio

2.4 Evidencia — CHANGELOG.md



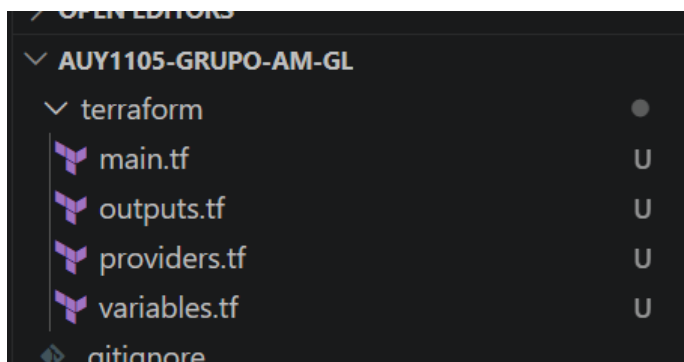
Figura 3: CHANGELOG.md documentando los cambios del proyecto por versión

3. Requerimiento 2 — Código de Infraestructura Terraform

Se desarrolló el código Terraform para desplegar infraestructura en AWS, estructurado en 4 archivos principales que siguen las mejores prácticas de organización de código IaC. Todos los recursos siguen la nomenclatura requerida: AUY1105-amgl-<tipo-recurso>.

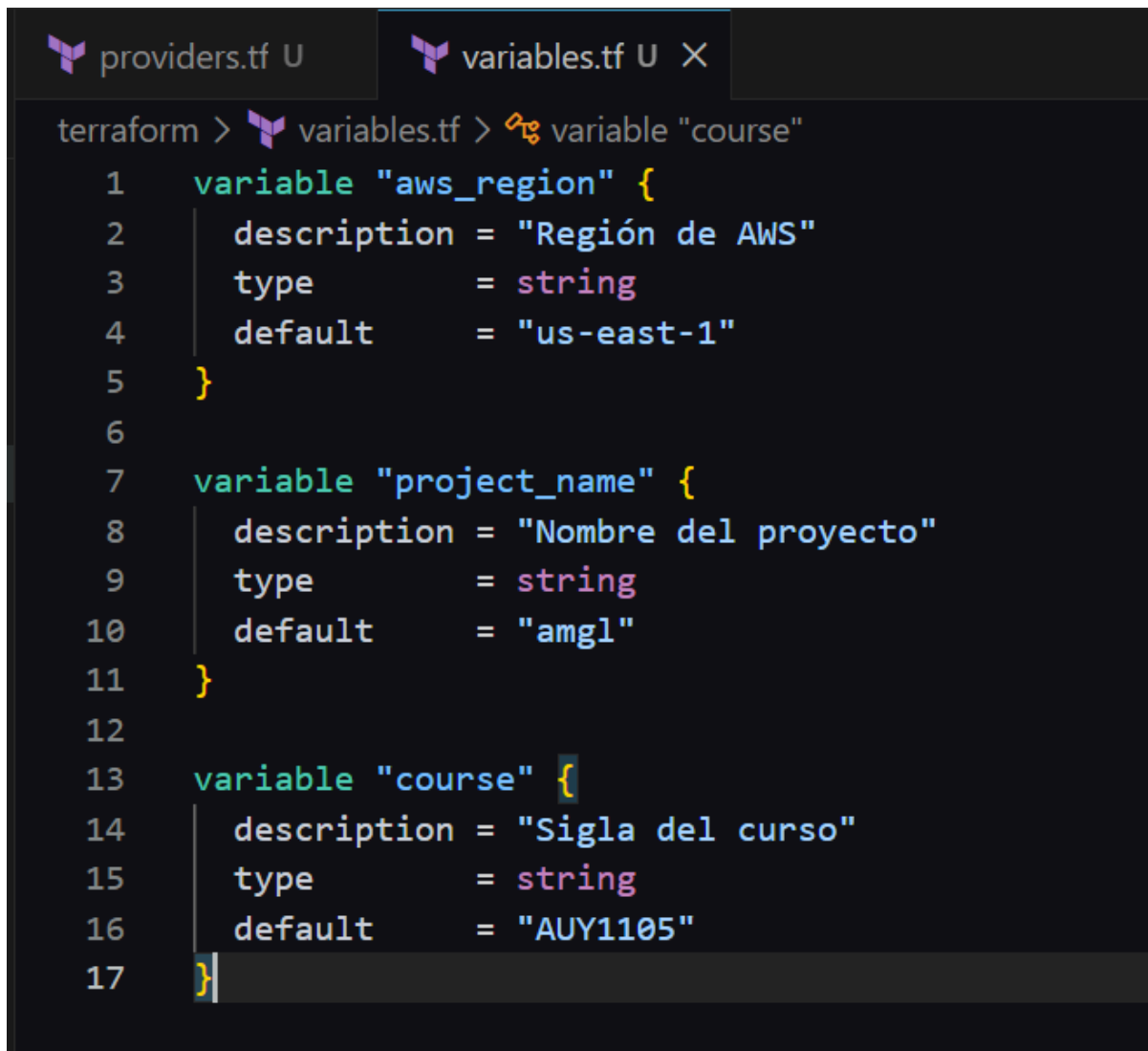
3.1 providers.tf — Configuración del Proveedor AWS

Define el proveedor de AWS con la versión 5.0 (última versión mayor disponible) y configura la región mediante una variable reutilizable. Esto garantiza compatibilidad y reproducibilidad del entorno.



3.2 variables.tf — Variables del Proyecto

Define las variables reutilizables del proyecto que permiten configurar el entorno sin modificar el código principal. Las variables definidas son: `aws_region` (us-east-1), `project_name` (amgl) y `course` (AUY1105).



```
providers.tf U  variables.tf U X
terraform > variables.tf > variable "course"
1  variable "aws_region" {
2      description = "Región de AWS"
3      type        = string
4      default     = "us-east-1"
5  }
6
7  variable "project_name" {
8      description = "Nombre del proyecto"
9      type        = string
10     default     = "amgl"
11 }
12
13 variable "course" {
14     description = "Sigla del curso"
15     type        = string
16     default     = "AUY1105"
17 }
```

3.3 main.tf — Recursos de Infraestructura

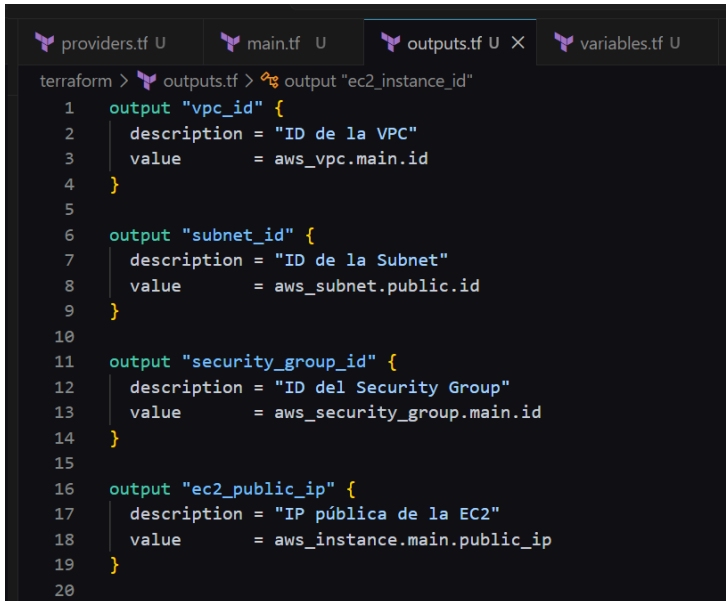
Define todos los recursos de infraestructura en AWS. Los recursos creados son:

Recurso AWS	Nombre	Configuración
aws_vpc	AUY1105-amgl-vpc	CIDR: 10.1.0.0/16
aws_subnet	AUY1105-amgl-subnet	CIDR: 10.1.1.0/24, AZ: us-east-1a
aws_security_group	AUY1105-amgl-sg	Ingress: SSH puerto 22 solo desde 10.0.0.0/8
aws_instance (EC2)	AUY1105-amgl-ec2	Ubuntu 24.04 LTS, tipo t2.micro

```
providers.tf U  main.tf U X  variables.tf U
terraform > main.tf > resource "aws_instance" "main"
1  # VPC
2  resource "aws_vpc" "main" {
3      cidr_block = "10.1.0.0/16"
4
5      tags = {
6          Name = "${var.course}-${var.project_name}-vpc"
7      }
8  }
9
10 # Subnet pública
11 resource "aws_subnet" "public" {
12     vpc_id      = aws_vpc.main.id
13     cidr_block   = "10.1.1.0/24"
14     availability_zone = "${var.aws_region}a"
15
16     tags = {
17         Name = "${var.course}-${var.project_name}-subnet"
18     }
19 }
20
```

3.4 outputs.tf — Outputs del Proyecto

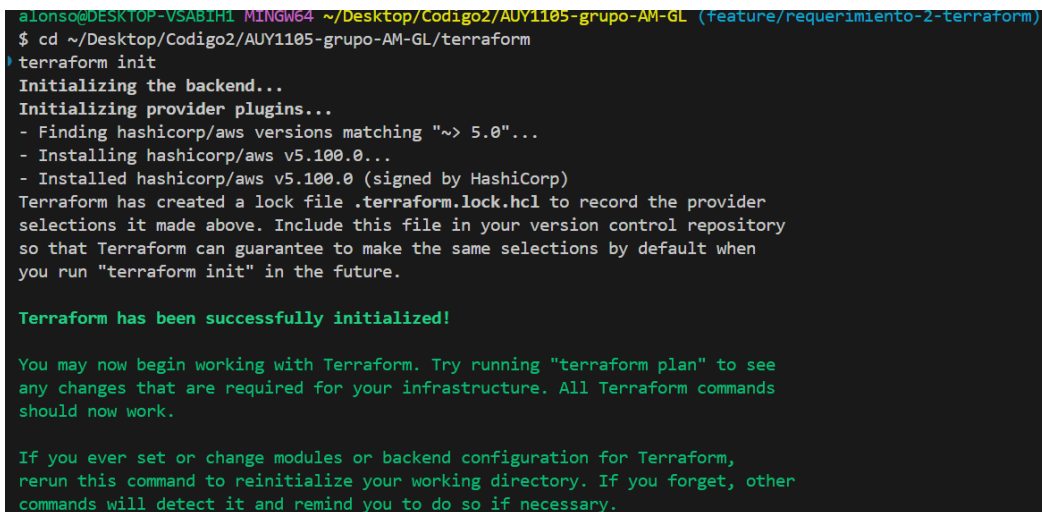
Define los valores de salida del proyecto que se muestran al finalizar la ejecución: `vpc_id`, `subnet_id`, `security_group_id`, `ec2_public_ip` y `ec2_instance_id`.



```
terraform > outputs.tf > % output "ec2_instance_id"
1  output "vpc_id" {
2      description = "ID de la VPC"
3      value       = aws_vpc.main.id
4  }
5
6  output "subnet_id" {
7      description = "ID de la Subnet"
8      value       = aws_subnet.public.id
9  }
10
11 output "security_group_id" {
12     description = "ID del Security Group"
13     value       = aws_security_group.main.id
14 }
15
16 output "ec2_public_ip" {
17     description = "IP pública de la EC2"
18     value       = aws_instance.main.public_ip
19 }
20
```

3.5 Validación del Código Terraform

Se ejecutaron los comandos `terraform init` y `terraform validate` para verificar que el código es válido y está correctamente configurado.



```
alonso@DESKTOP-VSABIH1 MINGW64 ~/Desktop/Codigo2/AUY1105-grupo-AM-GL (feature/requerimiento-2-terraform)
$ cd ~/Desktop/Codigo2/AUY1105-grupo-AM-GL/terraform
$ terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

```
alonso@DESKTOP-VSABIH1 MINGW64 ~/Desktop/Codigo2/AUY1105-grupo-AM-GL/terraform (feature/requerimiento-2-terraform)
● $ terraform validate
Success! The configuration is valid.
```

4. Requerimiento 3 — Automatización con GitHub Actions

Se implementó un workflow de GitHub Actions en el archivo `.github/workflows/terraform-ci.yml` que se activa automáticamente en cada Pull Request hacia la rama main. El pipeline ejecuta 3 etapas secuenciales de análisis de calidad y seguridad.

4.1 Estructura del Workflow

Etap a	Nombre	Herramienta	Descripción
1	Análisis Estático	TFLint	Analiza el código Terraform en busca de errores y desviaciones de mejores prácticas
2	Análisis de Seguridad	Checkov	Escanear vulnerabilidades de seguridad y configuraciones incorrectas
3	Validación Terraform	terraform validate	Valida la sintaxis y configuración del código Terraform

4.2 Configuración del Workflow

El workflow se configura con trigger `pull_request` hacia la rama main, garantizando que todo código revisado pase por el análisis antes de ser integrado. Las 3 etapas son secuenciales usando la directiva `needs`, de modo que si una etapa falla, las siguientes no se ejecutan. Además, consideramos los parámetros de `workflow_dispatch` para la ejecución manual del trigger y la creación de la configuración `.tflint.hcl` que es necesaria para el `-init`

feat: código Terraform - VPC, Subnet, Security Group y EC2 #2

Ready to merge Code

Open alonsoM10 wants to merge 1 commit into main from feature/requerimiento-2-terraform

Conversation 0 Commits 1 Checks 0 Files changed 5

+146



alonsoM10 commented now

Owner

Cambios realizados

- providers.tf: Configuración proveedor AWS v5.0 región us-east-1
- variables.tf: Variables del proyecto
- main.tf: Definición de VPC, Subnet, Security Group y EC2
- outputs.tf: Outputs de los recursos creados

Recursos creados

- AUY1105-amgl-vpc (CIDR 10.1.0.0/16)
- AUY1105-amgl-subnet (CIDR 10.1.1.0/24)
- AUY1105-amgl-sg (Solo SSH entrante)
- AUY1105-amgl-ec2 (Ubuntu 24.04 LTS t2.micro)

Reviewers

No reviews

Still in progress? [Convert to draft](#)

Assignees

No one—[assign yourself](#)

Labels

None yet

Projects

None yet

Milestone

No milestone

```
1   on:
2     pull_request:
3       branches:
4         - main
5     workflow_dispatch:
6
7   jobs:
8     analisis-estatico:
9       name: "1. Análisis Estático - TFLint"
10      runs-on: ubuntu-latest
11      steps:
12        - name: Checkout código
13          uses: actions/checkout@v4
14
15        - name: Crear directorio terraform si no existe
16          run: |
17            if [ ! -d "terraform" ]; then
18              mkdir -p terraform
19              echo "# Configuración básica de Terraform" > terraform/main.tf
20            fi
21
```

```
22        - name: Crear configuración de TFLint
23          run: |
24            cat > .tflint.hcl << 'EOF'
25            config {
26              module = true
27              force = false
28            }
29
30            plugin "aws" {
31              enabled = true
32              version = "0.40"
33              source  = "github.com/terraform-linters/tflint-ruleset-aws"
34            }
35
36            rule "aws_instance_invalid_type" {
37              enabled = false
38            }
39            rule "aws_instance_previous_type" {
40              enabled = false
41            }
42            EOF
```

```
49     - name: Inicializar TFLint
50       run: tflint --init
51       working-directory: terraform
52
53     - name: Ejecutar TFLint
54       run: tflint --format compact
55       working-directory: terraform
56
57   analisis-seguridad:
58     name: "2. Análisis de Seguridad - Checkov"
59     runs-on: ubuntu-latest
60     needs: analisis-estatico
61     steps:
62       - name: Checkout código
63         uses: actions/checkout@v4
64
65       - name: Ejecutar Checkov
66         uses: bridgecrewio/checkov-action@v12
67         with:
68           directory: terraform
69           framework: terraform
70           soft_fail: true
```



```

72     validacion-terraform:
73       name: "3. Validación Terraform"
74       runs-on: ubuntu-latest
75       needs: analisis-seguridad
76       steps:
77         - name: Checkout código
78           uses: actions/checkout@v4
79
80         - name: Instalar Terraform
81           uses: hashicorp/setup-terraform@v3
82           with:
83             terraform_version: "1.14.9"
84
85         - name: Terraform Init
86           run: terraform init -backend=false
87           working-directory: terraform
88
89         - name: Terraform Validate
90           run: terraform validate
91           working-directory: terraform

```

4.3 Resultado del Workflow — SUCCESS

Tras corregir el working-directory en el workflow, las 3 etapas ejecutaron exitosamente con un tiempo total de 43 segundos:

The screenshot shows the GitHub Actions interface for a workflow named "Terraform CI - Análisis de Calidad y Seguridad". The workflow run is titled "Título: fix: corregir working-directory en workflow TFLint #2" and has a status of "Success". It was triggered by a pull request from user "alonsoM10" and completed in a total duration of 43 seconds. The workflow file is "terraform-ci.yml" and it runs on "pull_request".

The workflow consists of three steps, all of which are marked as successful:

- 1. Análisis Estático - TFLint (5s)
- 2. Análisis de Seguridad - C... (17s)
- 3. Validación Terraform (14s)

The left sidebar shows the "Summary" tab selected, with links for "All jobs", "Usage", and "Workflow file". The right sidebar shows the "Summary" tab selected, with links for "Run details", "Usage", and "Workflow file".

5. Requerimiento 4 — Definición de Políticas OPA

Se implementaron 2 políticas de seguridad usando Open Policy Agent (OPA) en lenguaje Rego, almacenadas en la carpeta `policies/opa/`. Estas políticas garantizan que la infraestructura cumpla con los estándares de seguridad definidos.

5.1 Política 1 — Bloqueo de SSH Público (`politica_ssh.rego`)

Esta política deniega cualquier Security Group que permita acceso SSH (puerto 22) desde cualquier IP pública (0.0.0.0/0). Esto previene que la infraestructura quede expuesta a internet de forma inadvertida, cumpliendo con las mejores prácticas de seguridad de AWS y el principio de mínimo privilegio.

Lógica de la política: Si un Security Group tiene una regla ingress con `cidr_blocks = 0.0.0.0/0` y el rango de puertos incluye el puerto 22, la política deniega el recurso y retorna un mensaje de error descriptivo.

5.2 Política 2 — Control de Tipo de Instancia EC2 (`politica_ec2.rego`)

Esta política sólo permite la creación de instancias EC2 de tipo `t2.micro`. Cualquier intento de crear una instancia con un tipo diferente (`t2.small`, `t3.micro`, etc.) será denegado. Esto garantiza el control de costos y el cumplimiento de las políticas organizacionales.

Lógica de la política: Si una instancia EC2 tiene `instance_type` diferente a `t2.micro`, la política deniega el recurso y retorna un mensaje de error con el tipo de instancia no permitido.

5.3 Evidencia — Políticas OPA

```
providers.tf main.tf outputs.tf terraform-ci.yml politica_ssh.rego U politica_ec2.rego U varial
```

```
polices > opa > politica_ssh.rego
1 package terraform.policies.ssh
2
3 import future.keywords.if
4 import future.keywords.contains
5
6 deny contains msg if {
7   resource := input.resource_changes[_]
8   resource.type == "aws_security_group"
9   ingress := resource.change.after.ingress[_]
10  ingress.cidr_blocks[_] == "0.0.0.0/0"
11  ingress.from_port <= 22
12  ingress.to_port >= 22
13  msg := sprintf("ERROR: El Security Group '%v' permite acceso SSH público (0.0.0.0/0). Esto no está permitido.", [
14 }
```

```
polices > opa > politica_ec2.rego
1 package terraform.policies.ec2
2
3 import future.keywords.if
4 import future.keywords.contains
5
6 deny contains msg if {
7   resource := input.resource_changes[_]
8   resource.type == "aws_instance"
9   instance_type := resource.change.after.instance_type
10  instance_type != "t2.micro"
11  msg := sprintf("ERROR: La instancia EC2 '%v' usa tipo '%v'. Solo se permite t2.micro.", [resource.address, instance_type])
12 }
```

6. Requerimiento 5 — Buenas Prácticas de Revisión de Código

Se implementó un flujo de desarrollo mediante Pull Requests, donde cada requerimiento fue desarrollado en una rama separada y mergeado a main mediante PR documentado. Se realizaron 5 Pull Requests en total, todos con el merge realizado exitosamente.

6.1 Listado de Pull Requests

PR	Título	Estado	Rama
#1	feat: configuración inicial del repositorio - README, .gitignore y CHANGELOG	Mergeado	feature/requerimiento-1-repositorio
#2	feat: código Terraform - VPC, Subnet, Security Group y EC2	Mergeado	feature/requerimiento-2-terraform
#3	feat: workflow GitHub Actions - TFLint, Checkov y terraform validate	Mergeado	feature/requerimiento-3-automatizacion
#4	fix: corregir working-directory en workflow TFLint	Mergeado	fix/workflow-tflint
#5	feat: Requerimiento 4 - Políticas OPA	Mergeado	feature/requerimiento-4-politicas

6.2 Evidencia — Pull Requests en GitHub

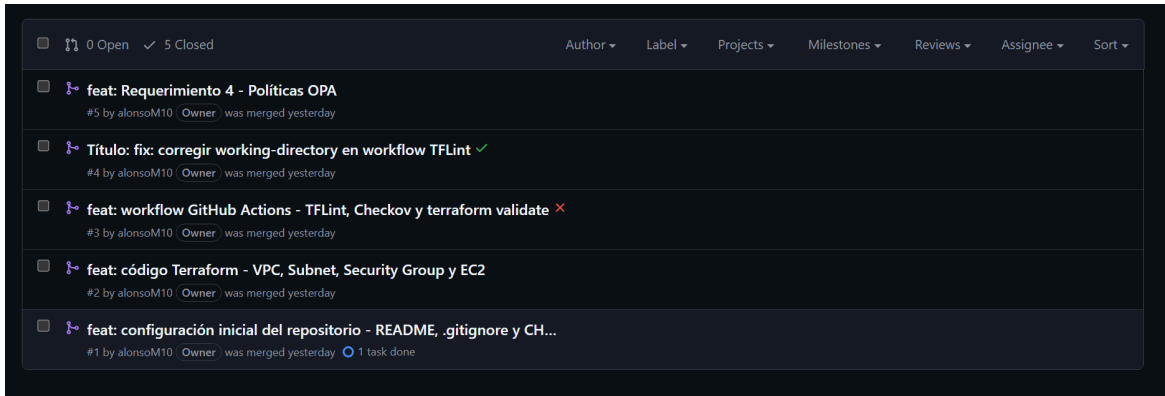


Figura 11: Los 5 Pull Requests mergeados en el repositorio GitHub

6.3 Descripción del Flujo de Trabajo

Cada Pull Request fue creado desde una rama de feature hacia main, siguiendo el flujo GitFlow.

Cada PR incluye:

- Título descriptivo con prefijo feat/fix según el tipo de cambio
- Descripción detallada de los cambios realizados y recursos creados
- Lista de archivos modificados verificados en la pestaña 'Files changed'
- Verificación del estado 'Able to merge' antes de crear el PR
- El workflow de GitHub Actions se ejecuta automáticamente al abrir el PR

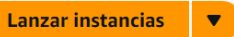
7. Evidencia de AWS en ejecución

```
Apply complete! Resources: 4 added, 0 changed, 0 destroyed.
```

Outputs:

```
ec2_instance_id = "i-03c10175cdef73e7a"  
ec2_public_ip = ""  
security_group_id = "sg-02022086d0ff72790"  
subnet_id = "subnet-05450681d3a8eed0a"  
vpc_id = "vpc-0ea114205dc817491"
```

```
PS C:\Users\gianc\Documents\2026 - DUOC\infra como codigo 2\AUY1105-grupo-AM-GL\terraform>
```

Instancias (1) Información    Estado de la instancia  Acciones  Lanzar instancias 

Conjuntos de filtros guardados

Elegir conjunto de filt... 

Q Buscar Instancia por atributo o etiqueta (case-sensitive)

< 1 > 

☐	Name 	ID de la instancia	Estado de la i...	Tipo de inst...	Comprobación de
☐	AUY1105-amgl-ec2	i-03c10175cdef73e7a	 En ejecución  	t2.micro	 2/2 comprobaci...

Grupos de seguridad (3) Información

  Acciones  Exportar los grupos de seguridad a CSV   Crear grupo de seguridad

Q Buscar grupos de seguridad por atributo o etiqueta

< 1 > 

☐	Name	ID de grupo de seguridad	Nombre del grupo de seguridad	ID de la VPC
☐	-	sg-0cb18f554ad7efb58	default	vpc-0ea114205dc817491
☐	AUY1105-amgl-sg	sg-02022086d0ff72790	AUY1105-amgl-sg	vpc-0ea114205dc817491

Detalles

ID de subred	ARN de subred
 subnet-05450681d3a8eed0a	 arn:aws:ec2:us-east-1:637423491510:subnet/subnet-05450681d3a8e

Detalles

ID de la VPC	Estado
 vpc-0ea114205dc817491	 Available

7. Conclusiones

La evaluación permitió implementar exitosamente un pipeline completo de CI/CD para infraestructura como código, integrando las principales herramientas de la industria. Los objetivos cumplidos fueron:

- Repositorio GitHub correctamente configurado con README.md, .gitignore y CHANGELOG.md, siguiendo las mejores prácticas de documentación.
- Código Terraform funcional que define VPC, Subnet, Security Group y EC2 en AWS, siguiendo la nomenclatura requerida AUY1105-amgl-<tipo-recurso>.
- Pipeline GitHub Actions con 3 etapas secuenciales (TFLint → Checkov → terraform validate) funcionando al 100% con estado SUCCESS.
- Políticas OPA implementadas en lenguaje Rego para controlar el acceso SSH público y el tipo de instancia EC2, garantizando el cumplimiento de los estándares de seguridad.
- Flujo de trabajo con 5 Pull Requests documentados y mergeados, evidenciando las buenas prácticas de revisión de código en equipo.

Como trabajo pendiente, se realizará el despliegue real de la infraestructura en AWS Academy mediante terraform plan y terraform apply, obteniendo evidencia visual de los recursos creados en la consola de AWS.